

Content

1. Introduction.....	2
System design; Flask; HTML; MYSQL; Combined Use	
2. Authentication.....	6
Code screenshot & implementation method	
3. E2EE.....	10
Requirements met & code display	
4.TLS&README.....	17
Implementation & Configure the certificate to make TLS feasible	
5.SCREENSHOT.....	22
Screenshots of software running	

Introduction

In this project, we developed and implemented a secure E2EE chat network application.

The system has the following characteristics:

1. Authentication and Lifecycle Management.
2. Implement a secure MFA mechanism based on passwords and OTP.
3. Encrypt communications between two users so that the server does not know the messages.
4. Protect communications in transit by configuring a modern TLS deployment.
5. Package a docker image of your web app.

We will introduce the implementation process of the system from the perspective of system technology architecture and security technology principles.

1. Download and install VirtualBox:

Go to <https://www.virtualbox.org/wiki/Downloads> and download the VirtualBox 7.10.14 platform packages according to your operating system.

VirtualBox 7.0.14 platform packages

- ➞ Windows hosts
- ➞ macOS / Intel hosts
- Linux distributions
- ➞ Solaris hosts
- ➞ Solaris 11 IPS hosts

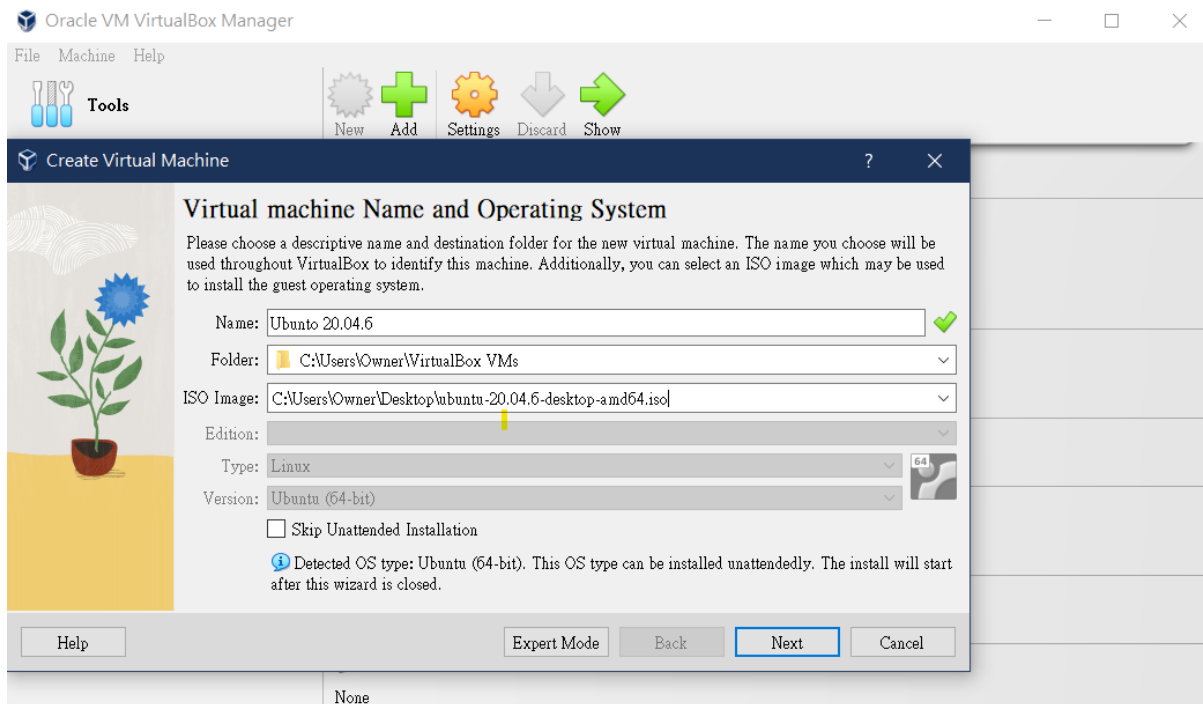
2. Download ubuntu-20.04.6-desktop-amd64.iso:

Go to <https://releases.ubuntu.com/focal/> and download the ubuntu-20.04.6-desktop-amd64.iso.

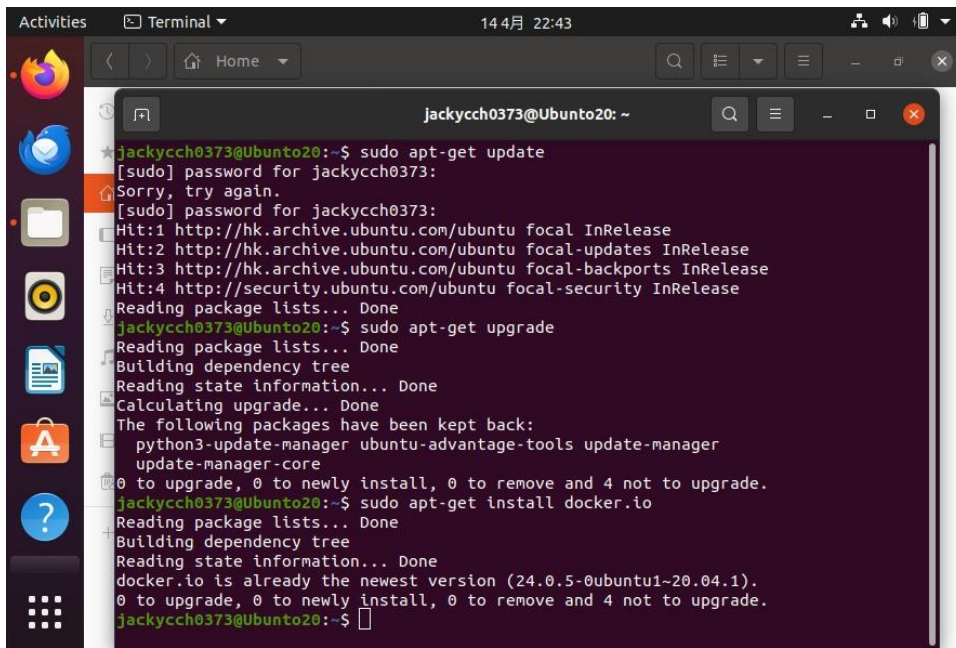
Name	Last modified	Size	Description
 Parent Directory		-	
 SHA256SUMS	2023-03-22 14:31	202	
 SHA256SUMS.gpg	2023-03-22 14:31	833	
 <u>ubuntu-20.04.6-desktop-amd64.iso</u>	2023-03-16 15:58	4.1G	Desktop image for 64-bit PC (AMD64) computers (standard download)
 ubuntu-20.04.6-desktop-amd64.iso.torrent	2023-03-22 14:31	325K	Desktop image for 64-bit PC (AMD64) computers (BitTorrent download)
 ubuntu-20.04.6-desktop-amd64.iso.zsync	2023-03-22 14:31	8.1M	Desktop image for 64-bit PC (AMD64) computers (zsync metafile)

3. Install Ubuntu onto Virtualbox:

<https://www.youtube.com/watch?v=yYbUO7Vtfus>; Skip to 0:47



4. **Ubuntu** download and unzip simple demo: [group5.zip](#) and go into folder:
chat
5. **Ubuntu Terminal:** sudo apt-get update
6. **Ubuntu Terminal:** sudo apt-get upgrade
7. **Ubuntu Terminal:** sudo apt-get install docker.io

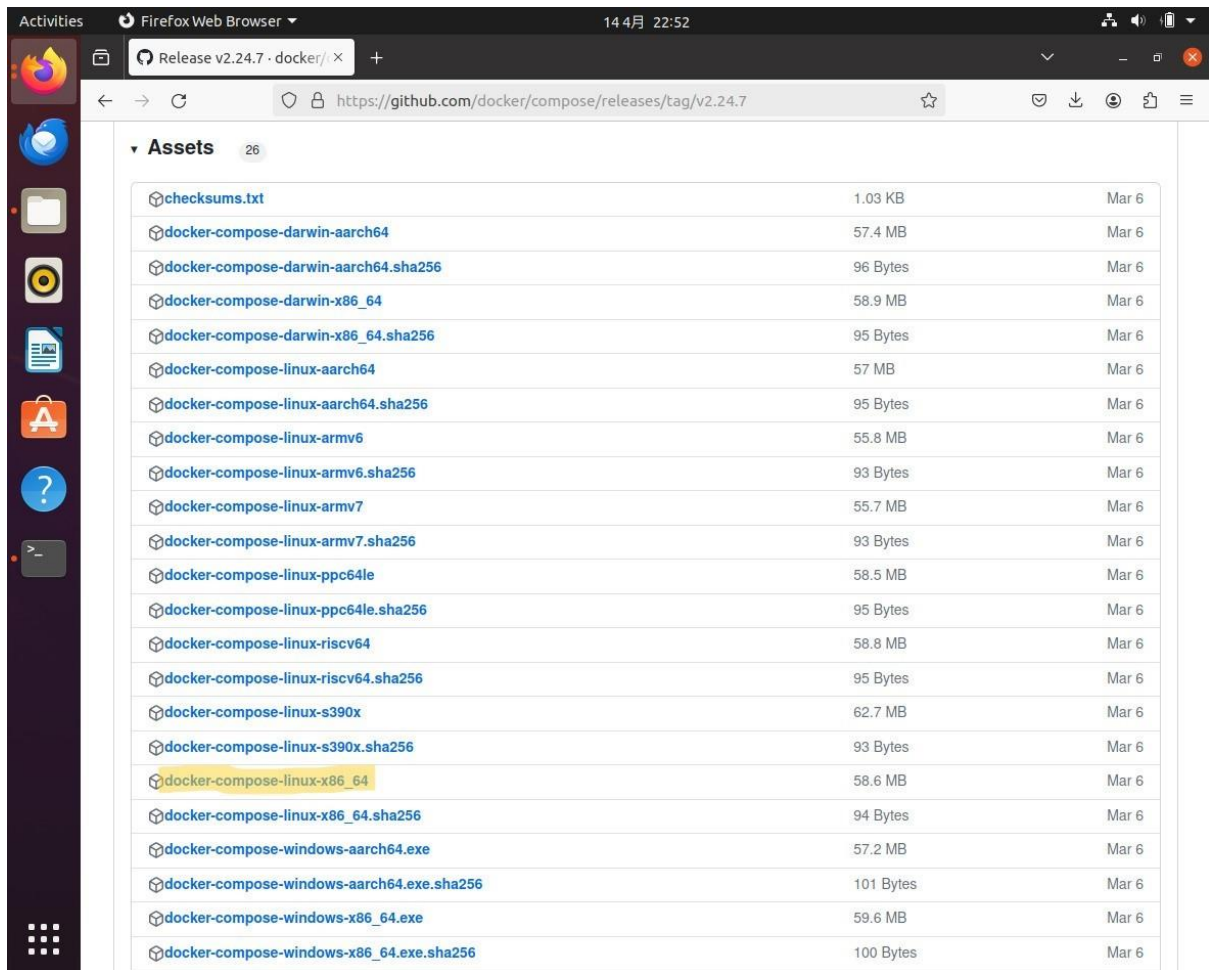
A terminal window titled 'jackycch0373@Ubuntu20: ~' is open on a Linux desktop. The terminal shows the execution of 'sudo apt-get update' and 'sudo apt-get upgrade'. The update process lists several packages from the 'focal' release that are in 'InRelease' state. The upgrade process shows that several packages (python3-update-manager, ubuntu-advantage-tools, update-manager, update-manager-core) have been kept back. Finally, 'sudo apt-get install docker.io' is executed, and the terminal reports that 'docker.io' is already the newest version (24.0.5-0ubuntu1~20.04.1). The desktop background is dark, and the terminal window has a dark theme.

```
jackycch0373@Ubuntu20:~$ sudo apt-get update
[sudo] password for jackycch0373:
Sorry, try again.
[sudo] password for jackycch0373:
Hit:1 http://hk.archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://hk.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://hk.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... Done
jackycch0373@Ubuntu20:~$ sudo apt-get upgrade
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
The following packages have been kept back:
  python3-update-manager ubuntu-advantage-tools update-manager
  update-manager-core
0 to upgrade, 0 to newly install, 0 to remove and 4 not to upgrade.
jackycch0373@Ubuntu20:~$ sudo apt-get install docker.io
Reading package lists... Done
Building dependency tree
Reading state information... Done
docker.io is already the newest version (24.0.5-0ubuntu1~20.04.1).
0 to upgrade, 0 to newly install, 0 to remove and 4 not to upgrade.
jackycch0373@Ubuntu20:~$
```

(If you do step 5-7 the first time, you will have to wait for a couple minutes.)

8. **Ubuntu** download docker-compose:

[https://github.com/docker/compose/releases/download/v2.24.7/docker](https://github.com/docker/compose/releases/download/v2.24.7/docker-compose-linux-x86_64)
[-compose-linux-x86_64](https://github.com/docker/compose/releases/download/v2.24.7/docker-compose-linux-x86_64)

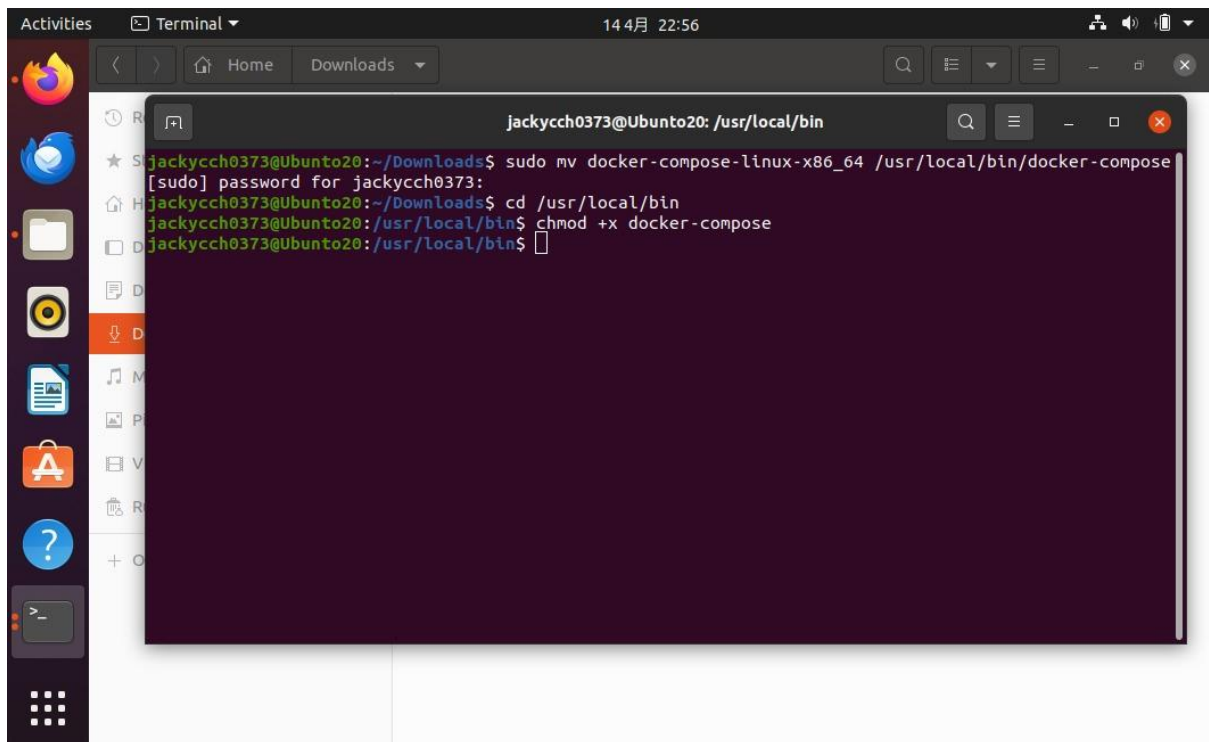


9. Open terminal in the folder you downloaded the docker-compose.

Command: `mv docker-compose-linux-x86_64 /usr/local/bin/docker-compose`

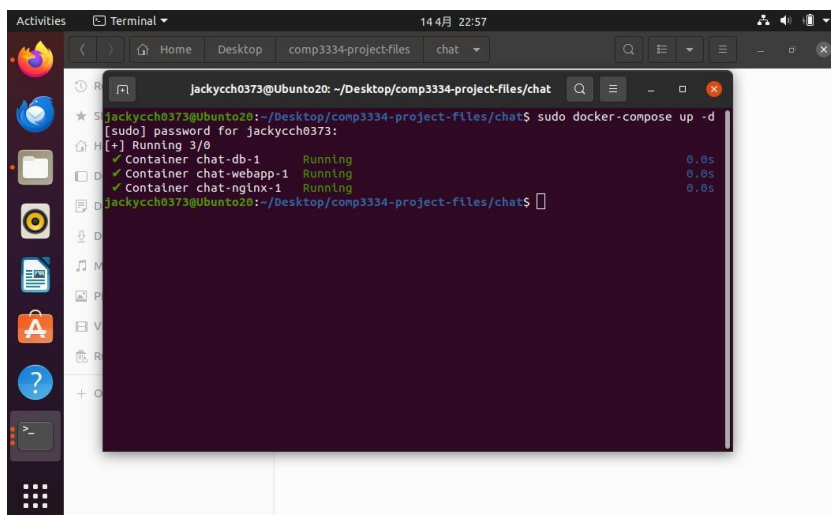
10. **Ubuntu Terminal:** `cd /usr/local/bin`

11. **Ubuntu Terminal:** `chmod +x docker-compose`



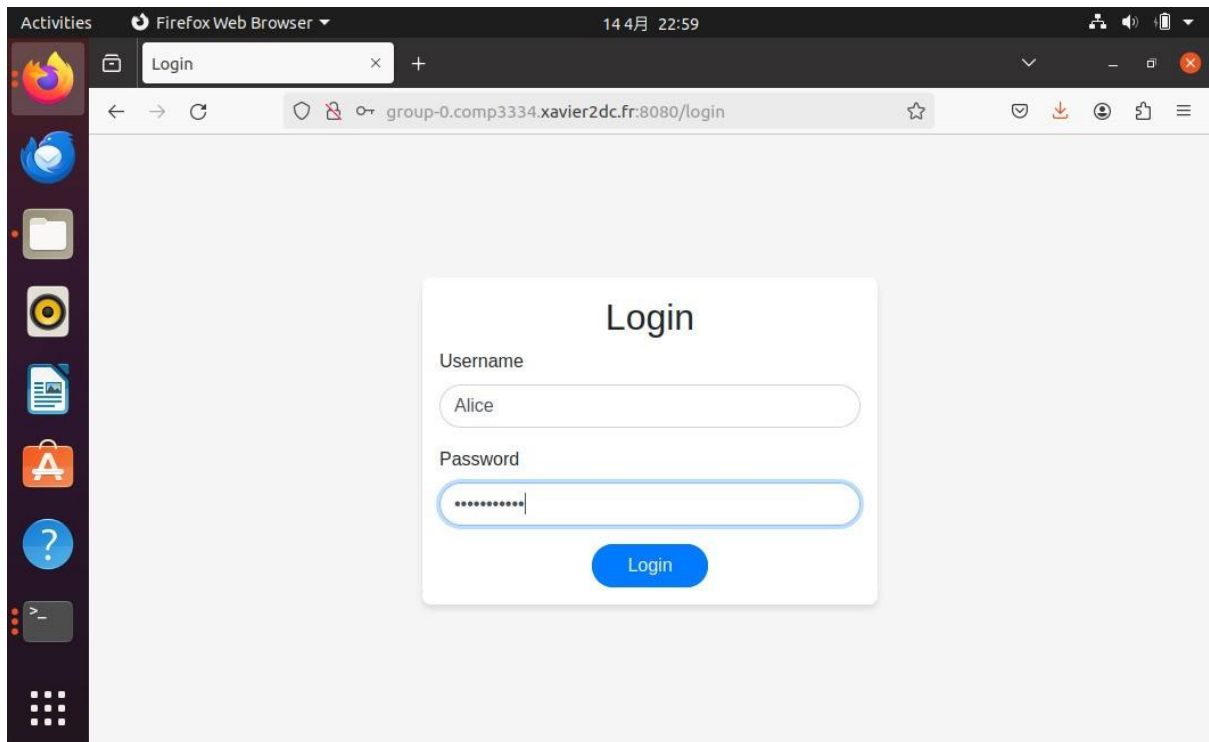
12. Open terminal in the simple demo folder: `comp3334-project-files -> chat`

(you downloaded it in step 4): `sudo docker-compose up -d`



13. Then enter this website using Ubuntu firefox: `http://group-`

5.comp3334.xavier2dc.fr:8334



System design

We developed the system using Flask+HTML+MYSQL technology architecture.

Flask, HTML, and MySQL play important roles in web development, and when combined, they can build feature rich web applications. The following are the basic principles of using these three technology stacks for development:

Flask

Role: Web Framework

Function:

1. **Routing management:** Define the mapping relationship between URLs and processing functions, and handle user HTTP login, registration, chat, and other requests
2. **Template rendering:** Combining with HTML templates to dynamically generate HTML pages
3. **Scalability:** Supports various plugins and extensions, such as database operations, form validation, etc.

Working Principle:

Working principle: When the web server receives a user's HTTP request, Flask will find the corresponding processing function based on the defined route, execute the function, and return a response. During this process, Flask may call database operations, template rendering, and other functions.

HTML

Role: Front end markup language

Function:

1. **Page structure:** Define the layout and structure of a webpage.
2. **Content display:** Display text, images, videos, and other content.
3. **Interactivity:** Implement interaction with users through technologies such as JavaScript.

Working Principle:

HTML files define the static content of web pages. Flask fills dynamic data into HTML templates through template rendering engines such as Jinja2, generates the final HTML page, and sends it to the user's browser.

MySQL

Role: Relational database management system

Function:

1. **Data storage:** Store data for application programs. For example, user tables, chat tables, login tables, etc
2. **Data Query:** Provide SQL language for data addition, deletion, modification, and querying operations.

Working Principle:

Flask interacts with MySQL databases through database extensions. When it is necessary to query or modify data, Flask will send an SQL statement to the MySQL server, which will execute the statement and return the result. Flask then executes the corresponding logic or renders the page based on these results.

Combined use

In actual development, Flask is responsible for handling user requests and generating responses, HTML is responsible for page display, and MySQL is

responsible for data storage and querying.

Authentication

Introduction

We provide multi-dimensional authentication mechanisms, including User-chosen Memorized Secret, Google OTP Authenticator, and password & Secret ciphertext storage. For authentication requirements, we follow selected sections from NIST Special Publication 800-63B.

We choose password to be the user-chosen memorized secret. To use the application, a user is required to register first. Extracting inputs from web page, we will check whether the user's input of account information is valid, including the account name's uniqueness within the database and the password's settings. A valid password must be at least 8 characters long, including uppercase and lowercase letters and numbers.

```

@app.route('/register', methods=['GET', 'POST'])
def register():
    error = None
    if request.method == 'POST':
        # get the register's username and password
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['con_password']

        if password != confirm_password:
            error = 'Password and Confirm Password are not same.'
            return render_template('register.html', error=error)

        if not check_password(password):
            error = 'Password settings not compliant: Passwords must be 8 characters long, contain at least one uppercase letter, one lowercase letter, and one digit.'
            return render_template('register.html', error=error)

        # to find the username exist
        if username_exists(username):
            error = 'user already exist!'
            return render_template('register.html', error=error)

```

```

def check_password(password):
    if len(password) < 8:
        return False
    if not re.search("[A-Z]", password):
        return False
    if not re.search("[a-z]", password):
        return False
    if not re.search("[0-9]", password):
        return False
    return True

```

```

def username_exists(username):
    # whether the username exists
    cur = mysql.connection.cursor()
    cur.execute("SELECT user_id FROM users WHERE username=%s", (username,))
    account = cur.fetchone()
    if account:
        return True
    return False

```

A 16-bit random salt is created and hashed with the new password. The hash is then hashed again with bcrypt and its random generated salt.

```

salt = binascii.hexlify(os.urandom(16)).decode('utf-8')

sha3_256 = hashlib.sha3_256()
sha3_256.update(password.encode('utf-8') + salt.encode('utf-8'))
hash_password = sha3_256.hexdigest()
bcrypt_hashed_password = bcrypt.hashpw(hash_password.encode('utf-8'), bcrypt.gensalt())

```

We choose to generate an OTP key to the user after accounts validation. We use PyOTP library to help with this process. We generate a secret key with `pyotp.random_base32()`, which returns a 32-bit string of uppercase letters and numbers. `pyotp.TOTP(secret_key)` is used to generate a TOTP with the secret key and current time.

```
# to generate the otp_key
secret_key = pyotp.random_base32()
totp = pyotp.TOTP(secret_key)
uri = totp.provisioning_uri(name=username, issuer_name='Chat App')
img = qrcode.make(uri)
qr_code_filename = f"{username}{secret_key}.png"
script_dir = os.path.dirname(os.path.abspath(__file__))
img.save(os.path.join(script_dir, 'otp_qrcode', f"{qr_code_filename}"))
session['qr_code_filename'] = qr_code_filename
```

Then, we generate a QR code with the username and the issuer name “Chat App” with an URI generated by `totp.provisioning_uri()`, set the image’s name and stored it in file `otp_qrcode`.

名稱	修改日期	類型	大小
__pycache__	10/4/2024 17:13	檔案資料夾	
otp_qrcode	10/4/2024 13:01	檔案資料夾	
sessions	10/4/2024 13:01	檔案資料夾	
static	10/4/2024 17:13	檔案資料夾	
templates	10/4/2024 22:31	檔案資料夾	
app	10/4/2024 17:13	PY 檔案	17 KB
db	10/4/2024 17:13	Yaml 來源檔案	1 KB
Dockerfile	10/4/2024 17:13	檔案	1 KB
requirements	10/4/2024 17:13	文字文件	1 KB

A client recovery key is also generated with a 48-bit random numbers. A server key is generated with the client key and a salt, and it’s hashed with sha3-256. Everything is saved in user database.

```

# to generate recovery key and encryption key
client_recovery_key = generate_client_recovery_key()
server_encryption_key = generate_server_encryption_key(client_recovery_key, salt)
session['recovery_key'] = client_recovery_key

# to save in db
cur = mysql.connection.cursor()
cur.execute("INSERT INTO users (username, password, salt, otp_key, encryption_key) VALUES (%s, %s, %s, %s, %s);",
            (username, bcrypt_hashed_password, salt, secret_key, server_encryption_key))
mysql.connection.commit()
cur.close()
return render_template('register.html', otp_code=secret_key)

if 'recovery_key' in session:
    recovery_key = session['recovery_key']
    session['recovery_key'] = None
    return render_template('register.html', recovery_key = recovery_key)

return render_template('register.html', error=error)

def generate_client_recovery_key():
    client_key = ''.join([str(os.urandom(1)[0] % 10) for _ in range(48)])
    return client_key

def generate_server_encryption_key(client_key,salt):
    combined_key = (client_key + salt).encode()
    # Use sha3_256 to hash the combined key
    server_key = hashlib.sha3_256(combined_key).hexdigest()
    return server_key

```

This is set to show the OTP code and the recovery key on the web page. The registration process is finished, and the web page will show the error checking.

```

return render_template('register.html', otp_code=secret_key)

if 'recovery_key' in session:
    recovery_key = session['recovery_key']
    session['recovery_key'] = None
    return render_template('register.html', recovery_key = recovery_key)

return render_template('register.html', error=error)

```

For the login process, there is one more function to check whether a user has more than 5 failed attempts within 5 minutes. If so, the user will be denied from the login and have to try later.


```

cur.execute("SELECT COUNT(*) FROM login WHERE login_ip = %s AND login_datetime > %s",
            (request.remote_addr, datetime.now() - timedelta(minutes=5)))

attempts_count = cur.fetchone()[0]
if attempts_count >= 5:
    error = 'Too many login attempts. Please try again later.'
    return render_template('login.html', error=error)

```

The remaining authentication process is similar to registration. We extract the user's salt from SQL database and use it to execute the hashed password with sha3-256 and the user input. The hashed password will then be compared with the stored hashed password in database. If they match, the attempt records with the user's current IP will be deleted and the user will be redirected to the chat page.

```

# query salt
cur.execute("SELECT salt FROM users WHERE username=%s", (username,))
salt = cur.fetchone()
if salt is not None:
    salt = salt[0]
    sha3_256 = hashlib.sha3_256()
    sha3_256.update(password.encode('utf-8') + salt.encode('utf-8'))
    hash_password = sha3_256.hexdigest()

    cur.execute("SELECT user_id, password, otp_key FROM users WHERE username=%s", (username,))
    account = cur.fetchone()
    stored_hashed_password = account[1]
    if bcrypt.checkpw(hash_password.encode('utf-8'), stored_hashed_password.encode('utf-8')):
        # reset the count of login

        totp = pyotp.TOTP(account[2])
        if totp.verify(otp_key):
            cur.execute("DELETE FROM login WHERE login_ip=%s", (request.remote_addr,))
            mysql.connection.commit()
            session['username'] = username
            session['user_id'] = account[0]
            return redirect(url_for('index'))
        else:
            return render_template('login.html', error='Invalid OTP')

```

If there is no salt for that user or the password is incorrect, the login process is denied, this attempt will be recorded, and the user will be redirected to the

login page again.

```
    else:
        # add to sql 'login'
        cur.execute("INSERT INTO login (login_ip, login_datetime) VALUES (%s, %s)",
                    (request.remote_addr, datetime.now()))
        mysql.connection.commit()
        error = 'Invalid Password'
    else:
        # add to sql 'login'
        cur.execute("INSERT INTO login (login_ip, login_datetime) VALUES (%s, %s)",
                    (request.remote_addr, datetime.now()))
        mysql.connection.commit()
        error = 'Invalid Salt'

    return render_template('login.html', error=error)

return render_template('login.html', error=error)
```

E2EE

Once users are logged in, secure chat messages between two users in a way so that the server cannot decrypt the messages.

We use the ECDH key exchange protocol to establish a shared secret between two users. Exchanged information during the key exchange can be sent through the server. We Choose P-384 as the underlying curve.

REQ1

The purpose of E2EE is to ensure the security of the chat messages between two parties (denoted as Alice and Bob respectively). We use ECDH with P-384

underlying curve in the function `EC_generate()` to generate a key pair. Then we use `crypto.subtle.exportKey()` method to export the public and private key from the key pair.

```
async function EC_generate() {
  try {
    const keyPair = await crypto.subtle.generateKey({name: "ECDH", namedCurve: "P-384"}, true, ["deriveKey", "deriveBits"]);
    const exportedPrivateKey = await crypto.subtle.exportKey("jwk", keyPair.privateKey);
    const exportedPublicKey = await crypto.subtle.exportKey("jwk", keyPair.publicKey);
    const privateKeyString = btoa(JSON.stringify(exportedPrivateKey));
    const publicKeyString = btoa(JSON.stringify(exportedPublicKey));
    const output = { privateKey: privateKeyString, publicKey: publicKeyString };
    return output;
  } catch (error) {
    console.error(error);
  }
}
```

After that, we can obtain the share secret using the `EC_sharedSecret()` function. We use the `crypto.subtle.deriveBits()` method to compute the share secret using P-384 ECDH and the public-private key pair.

```
async function EC_sharedSecret(privateKey, publicKey) {
  try {
    const privateKeyObj = JSON.parse(atob(privateKey));
    const publicKeyObj = JSON.parse(atob(publicKey));
    const importedPrivateKey = await crypto.subtle.importKey("jwk", privateKeyObj, { name: "ECDH", namedCurve: "P-384" }, true, ["deriveBits"]);
    const importedPublicKey = await crypto.subtle.importKey("jwk", publicKeyObj, { name: "ECDH", namedCurve: "P-384" }, true, []);
    const sharedSecret = await crypto.subtle.deriveBits({name: "ECDH", public: importedPublicKey, importedPrivateKey, 384});
    const sharedSecretString = btoa(String.fromCharCode(...new Uint8Array(sharedSecret)));
    return sharedSecretString;
  } catch (error) {
    console.error(error);
  }
}
```

REQ2

In the `deriveKeys()` function, we use the `crypto.subtle.deriveKey()` method to derive two 256-bit AES-GCM keys and two 256-bit MAC keys based on HKDF-SHA256. We differentiate each key by its 'info' parameters and the algorithm it uses (i.e. AES-GCM or HMAC) and the usage (i.e. encrypt/decrypt or sign/verify). Then we use `crypto.subtle.exportKey()` method to export the key

as a raw byte sequence and use new Uint8Array() method to create an unsigned 8-bit integer array to store it. For the purpose of the output, we do the padding and convert it to String.

```
const aesKey1_2_string = Array.from(new Uint8Array(await crypto.subtle.exportKey('raw', aesKey1_2)))
  .map(byte => byte.toString(16).padStart(2, '0'))
  .join('');
```

Each time the value of salt will be incremented by 1 in the newSalt() method in app.py to ensure the uniqueness of the salt.

REQ3

We encrypt the message and send it to the recipient in the sendMessage() function. After the key exchange, we get the AES key, MAC key and the counter from the local storage using localStorage.getItem() method.

```
const localStorage_key = '{{ sender_id }}_'+peer_id;
const AES_key = JSON.parse(localStorage.getItem(localStorage_key)).aes1_2;
const MAC_key = JSON.parse(localStorage.getItem(localStorage_key)).mac1_2;
var counter = JSON.parse(localStorage.getItem(localStorage_key)).counter;
```

We use the MAC key and counter to generate the IV. To protect the data integrity, the IV should be protected with HMAC-SHA256 in the encodeIV() function. To achieve this, we use crypto.subtle.sign() method to generate a digital signature for the data derived from the counter and derive the first 96 bits from the signature as the IV.

```

async function encodeIV(key, message) {
  const encoder = new TextEncoder();
  const keyData = encoder.encode(key);
  const messageData = encoder.encode(message);

  const cryptoKey = await crypto.subtle.importKey(
    'raw',
    keyData,
    { name: 'HMAC', hash: 'SHA-256' },
    true,
    ['sign']
  );

  const signature = await crypto.subtle.sign('HMAC', cryptoKey, messageData);
  const iv_string = btoa(String.fromCharCode(...new Uint8Array(signature)));
  return iv_string.slice(0,12);
}

```

The message should be encrypted in AES-GCM mode by `crypto.subtle.encrypt( )` method in the `encryptMessage( )` function. Then we combine the IV and the ciphertext in a new unsigned 8-bit integer array.

```

const ciphertext = await crypto.subtle.encrypt(
  { name: 'AES-GCM', iv: ivArrayBuffer },
  cryptoKey,
  data
);

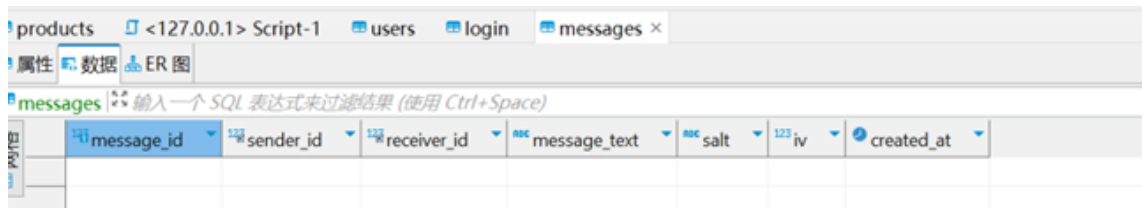
// Combine IV and ciphertext
const combined = new Uint8Array(12 + ciphertext.byteLength);
combined.set(ivArrayBuffer);
combined.set(new Uint8Array(ciphertext), 12);

```

After that, we create a payload containing `receiver_id`, encrypted message, salt and IV, then send the payload to the receiver using the `fetch( )` method. We increment the counter in the local storage by 1 since the counter represents the number of messages encrypted with the same key.

```
const dataSet = JSON.parse(localStorage.getItem(localStorage_key));
dataSet.counter = ++counter
await localStorage.setItem(localStorage_key, JSON.stringify(dataSet));
```

After the sender clicks to send the message, the message ciphertext will be stored in the database.



message_id	sender_id	receiver_id	message_text	salt	iv	created_at

In the displayMessage() function, the recipient uses the IV checker to keep track of the latest IV. The recipient verifies whether $IV_{ii} > IV_{ii-1}$ to prevent replay attacks.

```
//iv checker
if (message.iv > iv_checker){
    iv_checker = message.iv;
}else if (Number(message.salt) != Number(JSON.parse(localStorage.getItem(localStorage_key)).salt)){
    messageElement.textContent = `warning: replay attack detected`;
    messagesContainer.appendChild(messageElement);
}
```

REQ4

In the keyExchange() function, we use the localStorage.setItem() to store all the key material to local storage. So next time when the browser is reopened, all the key materials can be retrieved.

```
await localStorage.setItem(localStorage_key, JSON.stringify(dataSet));
console.log(JSON.parse(localStorage.getItem(localStorage_key)))
```

REQ5

In the `displayMessage( )` function, the recipient tries to decrypt the message. If it is successful, the text content will be displayed to the screen using the message container. Otherwise, if there is an error (e.g. Local Storage has been cleared, previous messages cannot be decrypted), a warning will be displayed on the screen using the `console.log( )` method.

```
const iv = await encodeIV(MAC_key, message.iv.toString());
try {
  const decrypted_message = await decryptMessage(message.message_text, AES_key, iv);
  messageElement2.textContent = `From ${sender} to ${receiver}: ${decrypted_message}`;
  messagesContainer2.appendChild(messageElement2);
} catch (error) {
  console.log()
}
```

REQ6

If the user clicks the 'refresh' button, the `refreshKeys( )` function will make a function call to `sendLog( )`, and the `sendLog( )` function will perform the key exchange again by making a function call to `keyExchange( )`. We set both `genSecret` and `genAES` variable in `keyExchange( )` function to `true`, so that the shared secret, AES and MAC key will be regenerated and store in the local

storage. A new salt is fetched by fetch() method.

```
function refreshKeys() {
  console.log('Refresh keys functionality will be implemented here.');
```

```
  // Placeholder for refresh functionality
  (async () => {
    if (peer_id == -1) return;
    await sendLog("Key change requirement")
    var key_salt
    // renew salt
    await fetch("/newSalt", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ user_id: peer_id})
    }).then(response => response.text()).then(salt => {
      key_salt = salt;
    }).catch(error => console.error('Error fetching messages:', error));
```

REQ7

In the decryptMessage() function, we check whether there is an existing ECDH key in the local storage using the localStorage.getItem () method. If no, we regenerate the ECDH key using EC_generate() function.

```
window.onload = function() {
  (async () => {
    if (localStorage.getItem("{ sender_id }_ECDH_key") != null) {
      const ECDH_key = JSON.parse(localStorage.getItem("{ sender_id }_ECDH_key"));
      console.log("MY PRIVATE KEY: "+ECDH_key.privateKey);
      console.log("MY PUBLIC KEY: "+ECDH_key.publicKey);
    } else {
      localStorage.setItem("{ sender_id }_ECDH_key", JSON.stringify(await EC_generate()));
    }
  })();
}
```

REQ8

In the `sendMessage( )` function, the network message to be sent is formatted in JSON in our own style.

```
const payload = {  
  receiver_id: peer_id,  
  message_text: encrypted_message,  
  salt: JSON.parse(localStorage.getItem(localStorage_key)).salt,  
  iv: counter  
};
```

REQ9

We use `console.log( )` method to print out all the crypto operations. So the information can be displayed to users clearly. For example, we use `console.log( )` to print out the payload containing `receiver_id`, `encrypted_message`, `salt` and `IV`.

```

function sendLog(message) {
  if (peer_id == -1) return; // Exit if no peer selected
  (async () => {
    await keyExchange();
    const localStorage_key = '{{ sender_id }}_'+peer_id;
    const AES_key = JSON.parse(localStorage.getItem(localStorage_key)).aes1_2;
    const MAC_key = JSON.parse(localStorage.getItem(localStorage_key)).mac1_2;
    var counter = JSON.parse(localStorage.getItem(localStorage_key)).counter;
    var iv = await encodeIV(MAC_key, counter.toString());
    const encrypted_message = await encryptMessage(message, AES_key, iv)
    const payload = {
      receiver_id: peer_id,
      message_text: encrypted_message,
      salt: JSON.parse(localStorage.getItem(localStorage_key)).salt,
      iv: counter
    };
    console.log(payload)
  });
}

```

REQ10

To prevent the XSS and CSRF attack, we set both the httponly and secure flag to true. Httponly means the cookie cannot be read by Javascript, which can prevent the XSS attack. Secure means the cookie will not be exposed to insecure connections, which can prevent the CSRF attack.

```

71 | 73 | @app.route('/')
72 | 74 | def index():
73 | -   if 'user_id' not in session:
74 | -       return redirect(url_for('login'))
75 | -   sender_id = session['user_id']
76 | -   return render_template('chat.html', sender_id=sender_id)
77 |
78 | +   if 'user_id' in session:
79 | +       sender_id = session['user_id']
80 | +       response = make_response(render_template('chat.html', sender_id=sender_id))
81 | +   else:
82 | +       response = make_response(render_template('login.html'))
83 | +
84 | +   csrf_token=generate_csrf_token()
85 | +   response.set_cookie('csrf', csrf_token, httponly=True, secure=True, samesite='Strict')
86 | +
87 | +   return response
77 | 88 |

```

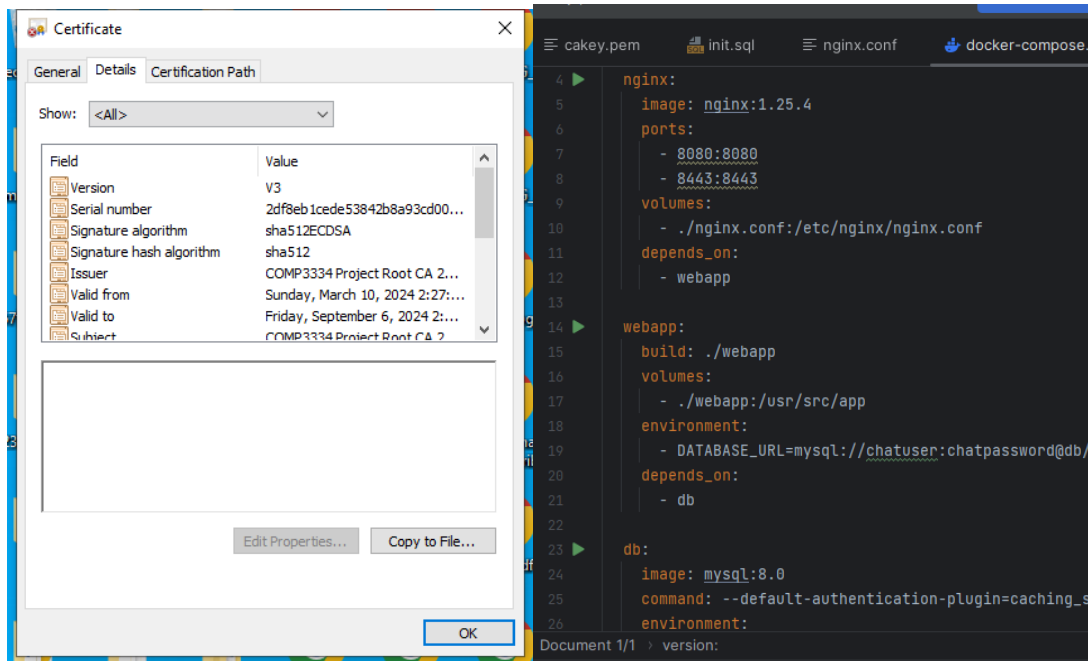

TLS

The TLS part involves certificate delivery, asymmetric encryption and cloud verification, so we will expand this part of the introduction through CA certificates and encryption algorithms.

By installing the certificate locally, you can pass security verification and achieve stable and secure end-to-end encryption.

The following sections will show the implementation principle of TLS and the key details of some certificates.

The TLS part corresponds to the following requirements:



REQ1 TLS version 1.3 only

we can check the TLS version from certificate. The version number is determined by project requirements, so it can implement more and more secure functions than in older versions.

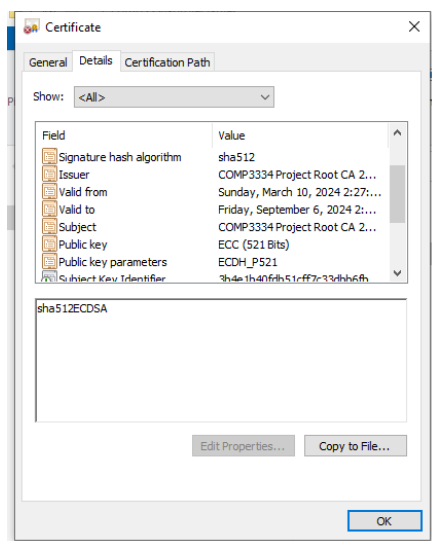
Version	V3
Serial number	2df8eb1cede53842b8a93cd00...

REQ2 x25519 Elliptic Curve Group only

This version of tls uses the x25519 Elliptic Curve Group encryption algorithm to achieve reliable encryption results.

REQ3 TLS CHACHA20 POLY1305 SHA256 cipher suite only

Our project uses the prescribed cipher suite and supports end-to-end encryption through subsequent procedures.



REQ4 No OCSP stapling (since you will use a self-signed CA certificate)

The CA certificate of our project is developed by the team members themselves to ensure compliance with no OCSP requirements and guarantee originality.

REQ5 HSTS for one week

HTTP Strict Transport Security is used in the project, and after a week of testing, it has been confirmed to be running correctly

REQ6 Details about TLS

Version	V3
Serial number	2df8eb1ced53842b8a93cd00...
Signature algorithm	sha512ECDSA
Signature hash algorithm	sha512
Issuer	COMP3334 Project Root CA 2...
Valid from	Sunday, March 10, 2024 2:27:...
Valid to	Friday, September 6, 2024 2:...

Subject	COMP3334 Project Root CA 2...
Public key	ECC (521 Bits)
Public key parameters	ECDH_P521
Subject Key Identifier	3b4e1b40fdb51cff7c33dbb6fb...
Basic Constraints	Subject Type=CA, Path Lengt...
Name Constraints	Permitted: [1]Subtrees (0..Ma...
Key Usage	Digital Signature, Key Encipher...
Thumbprint	a7334579096h36f503310d8f5

- Version V3 required

Version	V3
---------	----

- ECDSA public key over P-384
- SHA384 as hashing algorithm for signature

Serial number	2df8eb1ced53842b8a93cd00...
Signature algorithm	sha512ECDSA
Signature hash algorithm	sha512

- CA flag (critical): false
- Key Usage (critical) = Digital Signature algorithm included

Serial number	2df8eb1ced53842b8a93cd00...
Signature algorithm	sha512ECDSA
Signature hash algorithm	sha512
Issuer	COMP3334 Project Root CA 2...

- Extended Key Usage = Server Authentication

CN = COMP3334 Project Root CA 2024
OU = Department of Computing
O = The Hong Kong Polytechnic University
C = HK

- Include both Subject Key Identifier and Authority Key Identifier

Subject	COMP3334 Project Root CA 2...
Public key	ECC (521 Bits)
Public key parameters	ECDH_P521
Subject Key Identifier	3b4e1b40fdb51cff7c33dbb6fb...

- Validity period = 90 days

Valid from 3/10/2024 **to** 9/6/2024

REQ7 Website hosting link

<https://group-5.comp3334.xavier2dc.fr:8443/> (Authorization is required to access and is only used for server communication)

REQ8 All subdomains *.comp3334.xavier2dc.fr will redirect to 127.0.

Here is the redirecting page, which is used for convenience.

```
Permitted
[1]Subtrees (0..Max):
    DNS Name=comp3334.xavier2dc.fr
[2]Subtrees (0..Max):
    IP Address=127.0.0.1
    Mask=255.255.255.255
Excluded=None
```

REQ9

```
environment:
  - DATABASE_URL=mysql://chatuser:chatpassword@db/chatdb
depends_on:
  - db
db:
  image: mysql:8.0
  command: --default-authentication-plugin=caching_sha2_password
  environment:
    - MYSQL_DATABASE=chatdb
    - MYSQL_USER=chatuser
    - MYSQL_PASSWORD=chatpassword
    - MYSQL_ROOT_PASSWORD=rootpassword
  restart: always
  volumes:
    - ./db-data:/var/lib/mysql
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql

-----BEGIN EC PARAMETERS-----
BgUngQQAIw==
-----END EC PARAMETERS-----
-----BEGIN EC PRIVATE KEY-----
MIHcAgEBBEIAbkSeqKkKfZKkhw1LGfudcUVi02jwLRNRKyAHk8sZWeYdP0mg0HC+
jE/hExxksuhqfNmxxXULTc7JSh2DzEBf6SKgBwYFK4EEAC0hgYkDgYYABAEr4vP9
WBC3JyKZ+G00bhhySrB0KuL/GRHmJmferDln9Yfu2hk2bMJVonJzr5EoVHcciy8I
04eQIXHfNPveaxoKFgG0Z8RXDnLXXTpaS98N5iqKTS1G0z70sDJz5LeERejHZz6
66q7Fmug3+hQP057Fm4mLXWZ0CI+g1HFDiMwnwQcyw==
-----END EC PRIVATE KEY-----
```

TLS.README

This part is an introduction to how to use the software. Follow the below steps to run the chat program.

There are different operating methods based on linux and windows

COMP3334-GP5

3.8

All subdomains *.comp3334.xavier2dc.fr will redirect to 127.0.0.1

Linux:

Open a terminal.

sudo nano /etc/hosts

Add the following line at the end of the file:

127.0.0.1 *.comp3334.xavier2dc.fr

Save the changes and exit the text editor.

Windows:

Open File Explorer.

Navigate to the following directory:

C:\Windows\System32\drivers\etc\

Copy the hosts file to your desktop or any other location.

Open the copied hosts file using a text editor

Add the following line at the end of the file:

127.0.0.1 *.comp3334.xavier2dc.fr

Save the changes to the hosts file.

Copy the modified hosts file back to the original

location(C:\Windows\System32\drivers\etc), replacing the existing file.

Administrator permissions may be required.

3.10

To install a certificate on a PC

Certificate Link

Windows

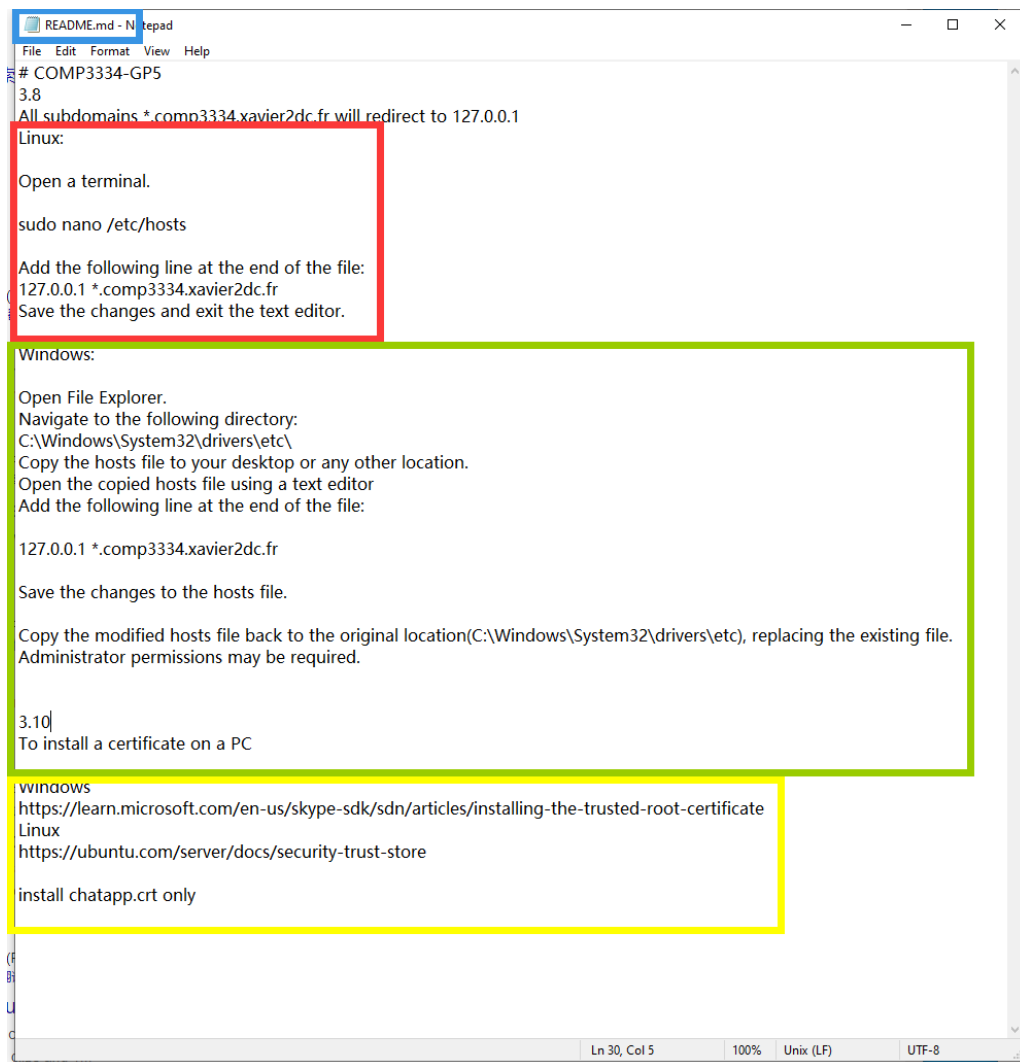
<https://learn.microsoft.com/en-us/skype-sdk/sdn/articles/installing-the-trusted-root-certificate>

Linux

<https://ubuntu.com/server/docs/security-trust-store>

trust cacert.crt

Install chatappsigned.crt (updated from chatapp.crt)

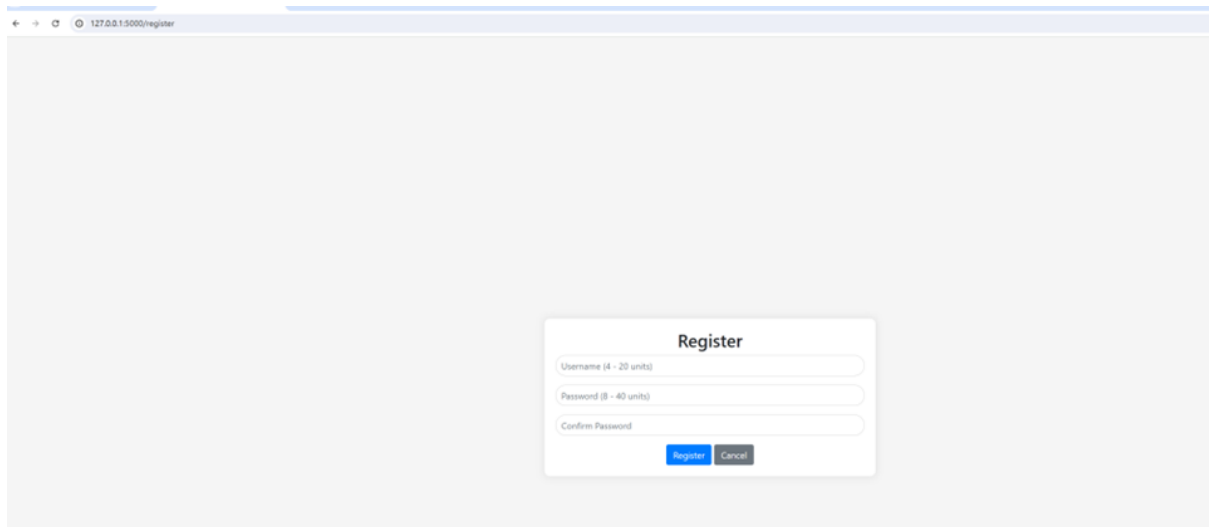


Screenshot

Introduction

this part is a screenshot of the software running. You can register and log in to your account and conduct encrypted chats safely.

Register



127.0.0.1:5000/register

Register

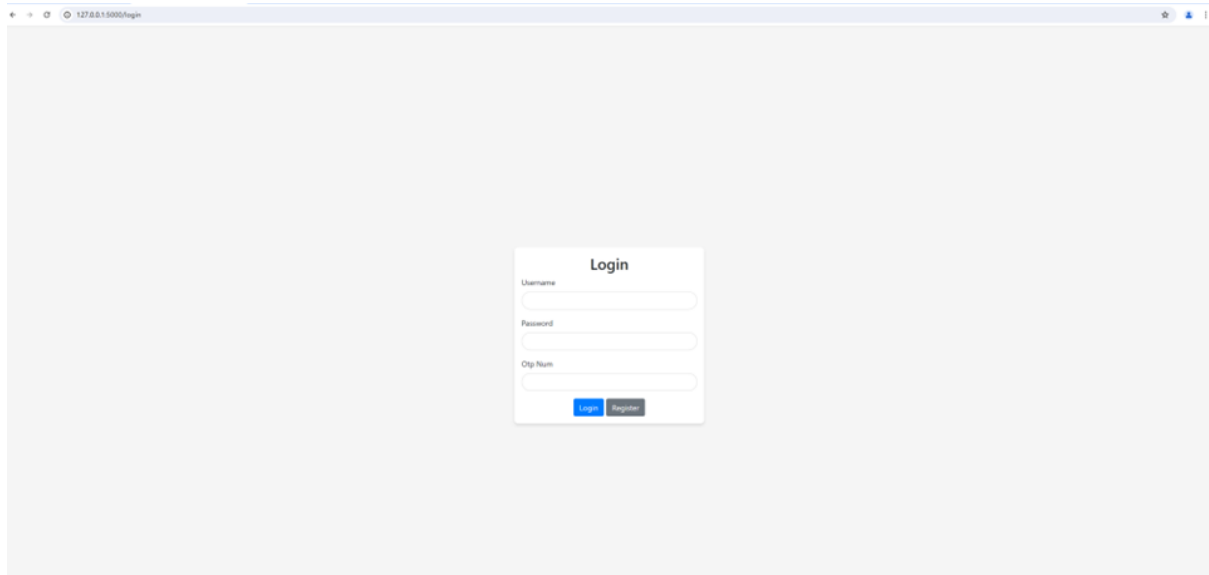
Username (4 - 20 units)

Password (8 - 40 units)

Confirm Password

[Register](#) [Cancel](#)

Login



127.0.0.1:5000/login

Login

Username

Password

Otp Num

[Login](#) [Register](#)

Chat

